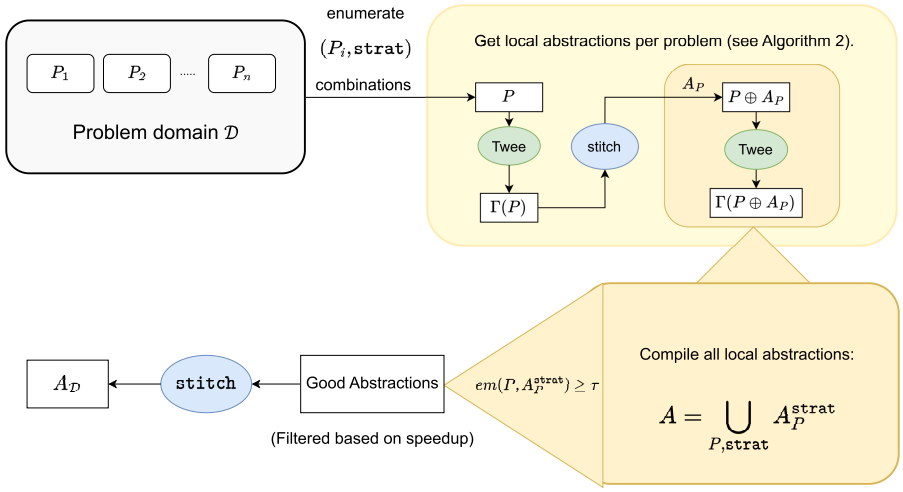

Fig. 1: Overview of domain abstraction generation via Twitch (Algorithm 1 and its subroutine Algorithm 2).

have derived a large collection of rewrite rules. Each of these derived rules $l \to r$ corresponds to an equation $l = r$ about the theory we are exploring. Many of these equations will be quite boring, but some will represent interesting lemmas. Our idea is to learn abstractions from only the most interesting lemmas.

In order to define interestingness, we first need some auxiliary definitions. For any lemma $l = r$, we shall denote by $T(l = r)$ the set of terms occurring in its proof, including the proofs of any sub-lemmas used in the proof. More precisely, the proof of $l = r$ is a chain $l = t_1 = \ldots = t_n = r$, where each step may be justified by a lemma $l_i = r_i$ having its own proof. We then define $T(l = r) = \{l, t_1, \ldots, t_n, r\} \cup \bigcup_i T(l_i = r_i)$.

For a term t, we define its weight $|t|$ as the number of function applications plus the number of unique variables occurring in t. We extend this to equations by defining $|l = r| = |l| + |r|$, and to sets of terms T by defining $|T| = \sum_{t \in T} |t|$. We then define the "interestingness" $s(l = r)$ of a lemma as follows:

$$s(l = r) := \frac{|T(l = r)|}{|l = r|^2}.$$

The intuition is that lemmas with higher scores are those that are simple statements requiring a long proof. Of course this does not necessarily coincide with what a human would consider interesting or useful, but we take it as a rough heuristic towards measuring these notions.

Given some partial proof of P_h, we sort the lemmas discovered in decreasing order of s. We then extract the first k lemmas $l_1 = r_1, \ldots, l_k = r_k$. What we call the *partial proof abstractions* are simply those we get by running Stitch on $\bigcup_{1 \leq i \leq k} T(l_i = r_i)$. We can then rerun Twee on P_h augmented with these abstractions.